Re-Evaluating Worst-Case Performance in Solving NP Hard Problems

Manas Asbanu - Supervisor Mark Stewart

The University of Adelaide

Abstract

Although NP-hard problems are not known to be solvable in polynomial time, they are often solved efficiently in practice. The contradiction raises questions regarding the usefulness of worst-case analysis for evaluating real-world algorithmic performance. This project investigates the discrepancy by empirically comparing the worst-case and expected performance of heuristic, approximation, and exact algorithms for the Travelling Salesman Problem. The findings support the importance of assessing algorithms beyond worst-case guarantees, showing the potential for more practical evaluation frameworks.

I. Introduction and Motivation

Background

To understand NP-hard problems, computational complexity must be considered. Computational complexity is the study of how efficiently computational problems can be solved by algorithms. These problems are grouped into decision, optimisation, or search types, and further classified into complexity classes based on their theoretical difficulty as follows.

- **P** problems can be solved in polynomial time.
- **NP** problems are those that a given solution can be verified in polynomial time.
- **NP-Hard** problems are at least as difficult as the hardest problems in NP, and are believed to lack polynomial-time algorithms for solving all instances.

These classifications provide a theoretical framework for evaluating the difficulty, time, and feasibility of developing a solution. This is especially important for resource utilisation in real-world applications. NP-hard problems occur frequently in practice, across domains like logistics, scheduling, and network-design. In such cases, optimal solutions are rarely necessary, and heuristic and approximation algorithms are used to produce good enough solutions efficiently.

The Problem

The standard theoretical model for assessing algorithm performance is worst-case analysis. It measures the maximum amount of time to compute a solution as a function of input size. This is typically represented by Big-O notation. For NP-hard problems, worst-case complexity is believed to be superpolynomial, growing at least exponentially. This is evident in The Travelling Salesman Problem (TSP), which has a time complexity of *O(n!)*. This renders exact algorithms impractical for large inputs.

However, real-world instances of NP-hard problems often follow patterns or structures, making them easier to solve in practice than worst-case suggests. Heuristic and approximation algorithms can often compute near-optimal solutions efficiently. This reveals a discrepancy between worst-case guarantees and empirical performance.

This project investigates that discrepancy. It explores how methods like approximation guarantees, heuristics, and smoothed analysis can better reflect practical performance compared to worst-case complexity. The goal is to quantify the usefulness of worst-case analysis, and identify when and why it fails to reflect real-world outcomes.

II. Literature Review

Approximation Algorithms

Opposed to exact algorithms, approximation algorithms provide near-optimal solutions with a guaranteed level of accuracy, expressed by the approximation ratio. This is particularly useful for solving NP-hard problems, where finding the optimal solution is unlikely in polynomial time. These algorithms are designed using common techniques including greedy methods, local-search, primal-dual methods, and relaxation (Vazirani, 2001).

The benefit of these algorithms are in the performance guarantees. A c-approximation algorithm ensures its solution *ALG(x)* is at most a factor of *c* within the optimal solution *OPT(x)*. For minimisation problems, this satisfies the following inequality where c represents the worst-case bound (Cornell University, 2012; Chekuri, 2018; Mouatadid, 2016).

$$ALG(x) \leq OPT(x) \leq c \ * \ ALG(x)$$

These bounds provide confidence when deploying approximation solutions to real-world instances. However, improving the approximation ratio often increases computational cost.

In regards to the Travelling Salesman Problem, a leap in research was made when Christophides presented the polynomial time 3/2-approximation algorithm in 1976. Known as Christofides' algorithm, it solves TSP in *O(n3)* time and generates a tour at most 1.5 times longer than the OPT (Genova, 2017). The algorithm has been improved in special cases of TSP, such as Euclidean instances, where polynomial-time approximation schemes have been developed (Arora, 1996). Additionally, Researchers from the University of Washington managed to improve the approximation ratio by

selecting spanning trees from a particular random distribution, although the margin of improvement was only 10-36 (Karlin, 2021). Despite these research efforts, Christofides' algorithm remains the best approximation ratio for the general case, and is a suitable baseline to benchmark and compare.

While advantageous, approximation algorithms still have limitations. By definition, they do not guarantee finding the optimal solution, which may prove inadequate for certain NP-hard problems. Additionally, they may become too computationally expensive or complex for large practical instances (Johnson, 1973). Furthermore, their design and evaluation focus on worst-case performance, which may not reflect structured real-world inputs. For example, generalisations like the Generalised TSP have emerged in fields ranging from medical logistics to waste management, where empirical performance often exceeds worst-case predictions (Kundakcioglu & Yurtkuran, 2023).

Heuristic Methods

Heuristic methods use information available or rules of thumb to produce near optimal solutions. Unlike approximation algorithms, heuristics do not provide approximation guarantees and emphasis is placed on the practical efficiency of the algorithm. This approach leads to better real-world application, where problems are too large for exact solutions and when approximation algorithms prove too slow or computationally expensive. The focus on average empirical behaviour of the algorithm means it can be tailored to specific problem instances.

A notable heuristic for TSP is the Nearest Neighbor. After starting at a random node, it visits the nearest unvisited city repeatedly, focusing on the locally optimal option opposed to the length of the entire tour. The result is a near optimal solution, calculated simply and efficiently. This makes it effective for scaling to large instances and a strong baseline heuristic to benchmark and compare (Hougardy, 2015).

The tradeoff between speed and performance using heuristic shortcuts may lead to potential errors or biases in results. Unlike approximation algorithms, the lack of an approximation guarantee may lead to suboptimal outcomes if not properly considered. This is especially important as performance varies depending on the problem instance. Despite the tradeoff, heuristics remain valuable in practice as industry prioritises speed and scalability over optimal accuracy.

Smoothed Analysis

Smoothed analysis combines worst-case and average-case analysis. It evaluates expected performance after introducing slight random perturbations to inputs. Worst-case analysis does not always reflect real-world application, whereas smoothed analysis provides a more realistic measure.

A research paper demonstrated how smoothed analysis explained the discrepancy between the theoretical and practical performance of the Simplex method. Where average-case performance is unconvincing due to the dominance of random inputs bearing little resemblance to the application domain (Spielman, 2004). They added random noise using a Gaussian distribution, which ensured small changes were more frequent than larger. The efficiency of the algorithm was then analysed based on the size of the input and the randomness, measured by the standard deviation of the Gaussian perturbations. The result was a polynomial smoothed complexity opposed to the exponential time worst-case. The same can be applied to other cases, and can be used to justify the practical success of heuristic and approximation algorithms for NP-hard problems.

Scope and Research Gap

This project focuses on the three approaches, approximation algorithms, heuristic methods, and smoothed analysis to understand how NP-hard problems are solved in practice. Approximation algorithms offer performance guarantees, but have a tradeoff between solution quality and runtime, limiting practicality for large instances. In contrast, heuristics are highly efficient and scalable, yet lack any performance guarantees. Smoothed analysis bridges this gap, explaining why algorithms with poor worst-case behaviour perform well on realistic and perturbed inputs.

By empirically evaluating the performance of approximation, heuristic, and exact algorithms on TSP under worst-case, average-case, and smoothed input conditions, this project aims to assess the real-world relevance of each. The core gap in research is the lack of empirical quantification of how misleading worst-case complexity can be, especially for practical problem instances. This study addresses that gap by comparing theoretical guarantees with observed performance, and exploring how that performance changes under different conditions. Deducing the usefulness of worst-case complexity in guiding the design, and implementation of algorithms for real-world NP-hard problems.

III. Methodology

Aim

The purpose of this project is to compare the worst-case with the expected performance of NP-hard problems via empirical analysis. And further quantify the distinction between them. The methodology is designed to produce systematic and reproducible performance data across different algorithms and input types. Which can be visualised to support an analysis of results. The approach separates the input generation, algorithm execution, metric collection, and visualisation to support modularity. This structured methodology provides a clear direction towards quantifying the distinction between worst-case analysis and expected real-world behaviour.

Problem Selection

The Travelling Salesman Problem (TSP) was selected for the project due to its relevance to real-world application and existing research and tools. TSP is a key NP-hard problem, where the goal is to locate the shortest route upon visiting a set of cities. There are a lot of existing resources and research available, and it occurs across industry in areas like logistics and route optimisation. Furthermore, it is supported by benchmarked datasets and exact, approximation, and heuristic solvers, making it an ideal choice for comparing performance across different algorithm classes.

Algorithm Selection

As discussed in the literature review, the following three algorithms were selected to span the different tradeoffs between runtime and solution quality.

- **Nearest Neighbor (NN)** A greedy heuristic with no approximation guarantee, sensitive to input structure
- **Christofides** A 1.5 approximation algorithm using MSTs and matchings
- **Bellman-Held-Karp** (BHK) An exact dynamic programming algorithm with exponential time complexity

The selection enables comparison of empirical behaviour against theoretical expectations and adverse inputs, especially regarding the relevance of worst-case analysis across algorithm types.

Performance Metrics

The performance across algorithm and input types are measured according to execution time and solution quality. The solution quality is defined by the relative error, which in this context is the percentage by which a tour exceeds the optimal. The expected tour is calculated using BHK.

$$err = \left|\frac{actual - expected}{expected}\right|$$

These metrics show the tradeoff between quality and efficiency, and allow for appropriate comparison to theoretical guarantees based on the algorithm behaviour.

Input Categories

The input types are categorised as random, worst-case, and smoothed. The random inputs are uniformly randomly generated coordinates in 2D space and represent expected-case performance. Whereas the worst-case inputs are handcrafted to degrade performance according to behaviour. NN greedily picks the closest city at each step, thus a rectangular grid pattern is used where the opposite side is closer than the adjacent node. This results in a suboptimal zigzag path, opposed to a clean round trip.

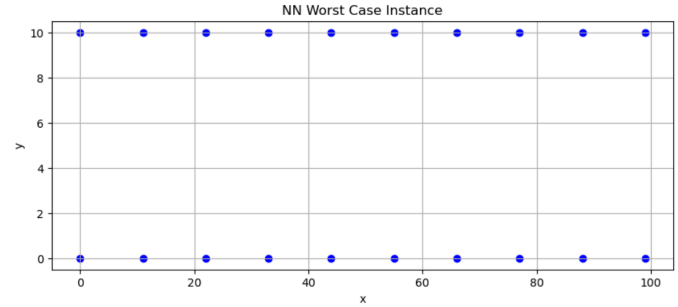

Figure 1: Worst case input structure for NN

Christofides uses global information on the cities making it difficult to trick. It computes a minimum spanning tree (MST) and minimum-weight perfect matching on odd degree vertices of the MST. It then produces a Eulerian multigraph, from which it takes a shortcutting Euler tour to get a Hamiltonian circuit. The input consists of two parallel lines with distant spikes. The MST includes the long edges connecting the spikes, which creates odd-degree vertices inefficiently matched, leading to high-cost during perfect matching. The resulting shortcutting Euler tour cannot mitigate the suboptimal edges and produces a poor approximation of the optimal Hamiltonian cycle (Christofides, 2022).

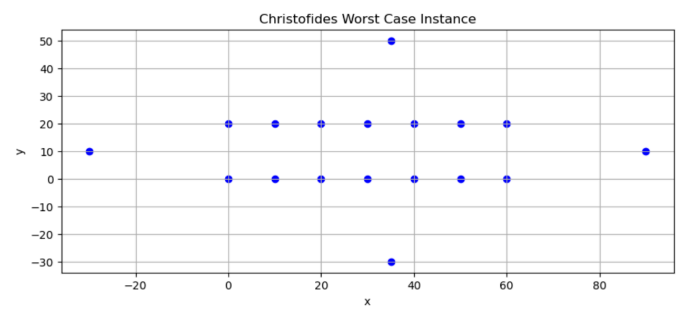

Figure 2: Worst case input structure for Christofides

BHK computational time grows exponentially with the number of vertices. Thus, dense graph layouts with many close routes should increase computational branching. The smoothed inputs are generated by applying Gaussian noise to the worst-case inputs, to evaluate robustness to small perturbations. These measure the fragility of the worst-case behaviour under randomness.

Execution Procedure and Data Collection

The general procedure for execution and data collection incorporated a unified execution harness. It was built to generate all input types, run each algorithm with safe error handling, and log output. Multiple trials were run for random and smoothed inputs, whereas deterministic worst-cases and external input data sets were only executed once per input size. Due to the limitations of BHK, input sizes iterated between 5-20. All results were saved to CSV for reproducibility and analysis.

Statistical Analysis

The statistical analysis followed a general plan applied to each algorithm class. With the goal of quantifying the usefulness of worst-case analysis based on the empirical results.

1. Exploratory Data Analysis (EDA)

Initially, an exploratory data analysis was performed on the results to understand the distribution and variability of algorithm performance. Distributions were plotted of runtime and relative error across different perturbation levels, starting nodes, and both structured and random instances. Boxplots were used to visualise spread and detect high variance, and summary statistics including mean, standard deviation, and IQR were calculated to assess consistency and outliers.

2. Linear Regression Analysis

Multiple linear regression was then performed to quantify the relationship between input characteristics and performance using the following model.

$$Relative\ Error\ or\ Runtime \sim sigma\ +\ Input\ Type\ +\ Starting\ Node$$

The results were interpreted to assess the statistical significance of each predictor using p-values, the overall model fit using R squared, and coefficient size as an indication of algorithm sensitivity to the respective factor.

3. Quantifying Results

Based on the regression results and summary statistics, a sensitivity score could be constructed for the algorithm. Where high coefficient magnitudes indicate high sensitivity, and high R squared imply more predictable variation due to input changes. The expression chosen to define this score is the sum of statistically significant coefficients, and is shown below.

$$SS = \sum |significant\ coefficients|$$

The performance variability was quantified using the standard deviation and IQR recorded during the EDA, providing an empirical basis for assessing algorithm stability.

$$VS = \frac{SD+IQR}{2}$$

The worst-case gap was calculated to measure the discrepancy between theoretical and observed performance as follows.

$$WCG = worst\ case\ bound\ -\ average\ observed\ error$$

4. Cross Comparative Analysis

Each algorithm result was then compared using the metrics quantified above to evaluate how reflective worst-case analysis is of actual performance. By comparing the results, any trends influencing when and why worst-case bounds are useful or misleading could be identified. These findings enable an empirical assessment on the usefulness of worst-case analysis across algorithm types to be made.

5. Amendments for BHK and Christofides

While Nearest Neighbor followed the analysis framework above, BHK and Christofides required methodological adjustments due to their behaviour. BHK underwent regression for log-transformed theoretical runtimes to assess scaling behaviour. Whereas Christofides approximation quality was evaluated using one-sample t-tests and regression analysis on error against input size. These alternative analyses are described in detail in the Results section.

IV. Experimental Setup

Libraries and Environment

Python version 3.11 was the kernel used for the Jupyter Notebook, with Microsoft Excel for the multiple linear regression model. In terms of libraries, the points below list each and its respective purpose.

- numpy - *coordinate generation and distance matrix computation*
- matplotlib and seaborn - *result visualisation*
- pandas - *data logging, manipulation and CSV exporting*
- time - *time.perf_counter() runtime measurement*
- os and csv - *file system operations and results saving*
- pyCombinatorial - *python implementations of TSP solving algorithms and util.build_distance_matrix()*
- random - *randomising instance generation and smoothed analysis*

These libraries are standard practice and are both reliable and optimal for numerical computation and plotting. Python is effective for readability and rapid prototyping for the short project timeframe. While pyCombinatorial contains a wide range of TSP algorithms with clear documentation and utility functions including matrix building for input.

Input Generation

Functions were defined for each type of input generation. They received input size as a parameter and returned an array of coordinates that were converted to a distance matrix for algorithm input via the pyCombinatorial and numpy utility functions. A seed parameter was returned each trial to ensure reproducibility.

```
distance_matrix = util.build_distance_matrix(np.array(coords))
```

For random generation, coordinates were generated on a 1000 by 1000 2D plane to reflect a range of distances while remaining bounded. The uniform distribution ensured a random selection while allowing identical reruns using the seed.

```
coords = np.random.rand(n_cities, 2) * 1000
```

Worst-case input functions constructed an array of coordinates that best fit the intended worst-case structure given the input size. This involved distributing the input size

across the various features and generating each one at a time, appending the results to a coordinate array. Each coordinate set was shuffled to enable the random starting node condition. The functions were verified by plotting the coordinates using matplotlib, ensuring an accurate input structure. Specific implementation is described in the github repository in the appendix.

The smoothed input generation added gaussian noise to each coordinate, where sigma was initialised at 1 and was varied to test sensitivity and fragility.

```
coord + np.random.normal(0, sigma)
```

Execution Procedure and Data Collection

The variables required for statistical analysis are listed below, note that some variables only apply to certain algorithms. All CSV files were exported and imported for cleaning using pandas.

- Algorithm
- Input type
- Input size
- Seed
- Relative error
- Starting condition (NN)
- Sigma
- Optimality gap

Visualisation and Summary Statistics

All plots and summary statistics were created using seaborn and matplotlib in separate Jupyter notebooks, where the csv data was imported using pandas into a dataframe. The multilinear regression was implemented in Microsoft Excel using the Data Analysis ToolPak add in.

V. Results and Analysis

V.1 Initial Combined EDA

The combined EDA verifies that the observed behaviour of each algorithm aligns with theoretical expectations. The plotted runtimes confirm these trends. BHK grows exponentially with input size, while NN remains near constant and Christofides shows moderate polynomial growth.

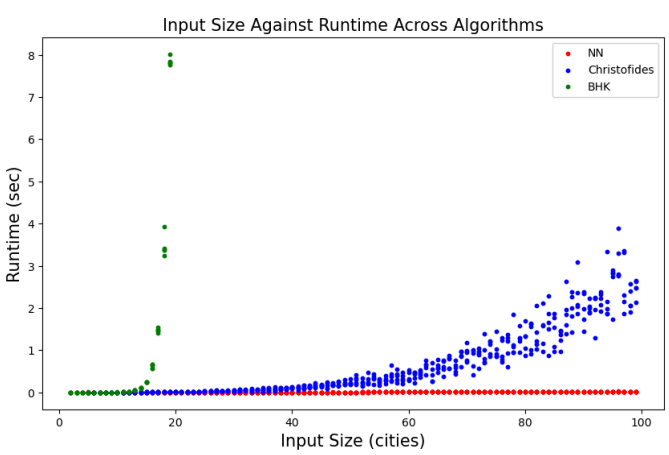

Figure 3: Scatterplot of input size against runtime for each algorithm

Relative error shows the inverse of runtime, supporting the theorised tradeoff between solution quality and efficiency. BHK maintains zero error, NN increases steadily with high variability, whereas christofides show minimal growth with occasional spikes.

Figure 4: Line graph relative error against input size for each algorithm

V.2 Per-Algorithm Analysis

V.2.1 Nearest Neighbor Sensitivity Regression Analysis

After measuring performance over different input conditions, multilinear regression was performed. The results of the regression were supported by EDA plots. As predicted, the results showed input condition had no effect on runtime, thus relative error was used as a performance benchmark. With an R square value of 0.708, the model explained roughly 71% of the variance in relative error, indicating the selected features were meaningful in predicting solution quality.

Regression Statistics			
Multiple R	R Square	Adjusted R Square	Standard Error
0.841664	0.708399	0.687895911	0.060545925

	Coefficients	*Standard Error*	*t Stat*	*P-value*
Intercept	-0.393054	0.11355459	-3.46136	0.00073
cities	0.0533784	0.01558645	3.42467	0.00083
sigma	-0.040828	0.04916995	-0.83034	0.40789
start_random	0.1029923	0.01563289	6.58818	1E-09
start_worst	0.211722	0.01563289	13.5434	1.8E-26
input_smoothed	0.1660004	0.05374497	3.08867	0.00247
input_worstcase	0.2416403	0.02068037	11.6845	6.5E-22
cities_squared	-0.001528	0.00051655	-2.95788	0.00369
sigma_squared	0.0031873	0.00742677	0.42917	0.66852
cities_sigma	-0.00048	0.00103518	-0.46347	0.64381

Figure 5: Regression statistics for NN input variations

Input Sensitivity to Starting Conditions

The error increased linearly with input size at 0.053 per city. Although the negative quadratic term -0.0015 for cities squared suggests this levels out at higher sizes. The most influential variable was the starting condition, where a

worst-case starting node added 0.212 to error and a random starting node added 0.103. Given the high significance, it shows the performance of NN is highly sensitive to starting position. Where poor initialisation can sometimes more than double the error compared to an optimal start. This is visualised by the boxplot below.

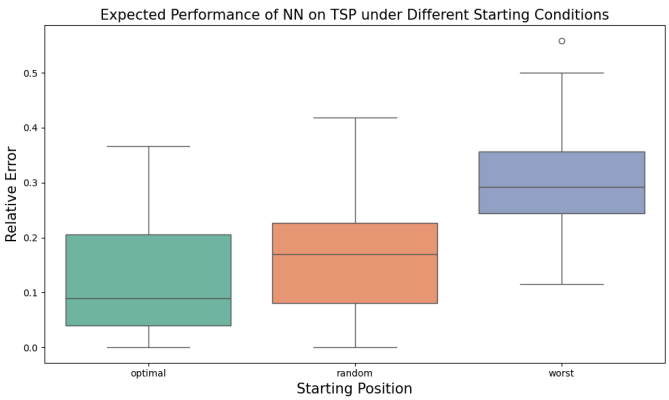

Figure 6: Boxplot of relative error for NN expected performance under different starting conditions

Sensitivity to Input Structure

For input type sensitivity, the worst-case inputs caused a 0.242 increase in relative error compared to expected, whereas the smoothed inputs also elevated error but to a lesser extent of 0.166. As both input types originated from the same layout, the difference reflects how much the gaussian perturbations mitigated the damaging structure of the worst-case inputs. Calculating this difference produced 0.076, and is better visualised below.

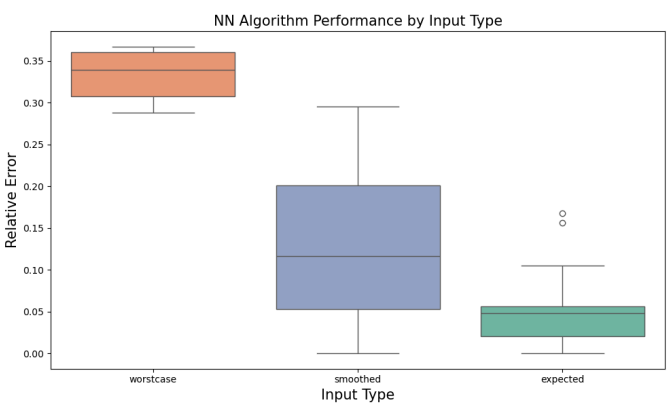

Figure 7: Boxplot of relative error for different NN input types

Notably, the coefficient of sigma was not statistically significant with a high 0.408 p-value. This suggests sigma values broke the adversarial structure regardless of size, leading to a non-linear or threshold-like change. The consistent drop in relative error when adding noise is visualised below, and supports the view that worst-case behaviour is fragile for NN.

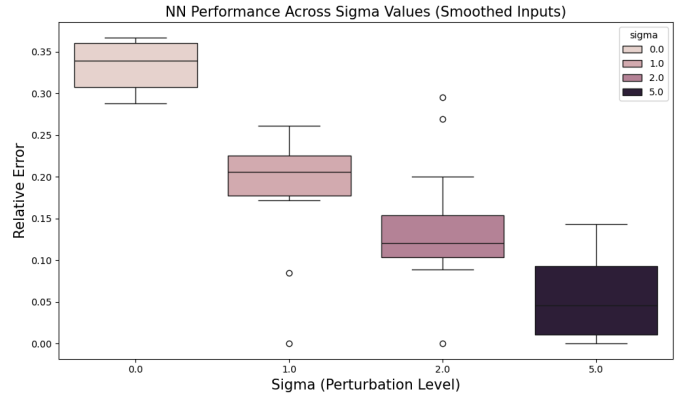

Figure 8: Boxplot of relative error for different NN sigma values

Quantifying Results

Based on the results, the quantifying metrics were calculated and entered into the table below to be later used for cross comparison. The theoretical worst-case of 1.0 was used for NN, which occurs upon miss-classifying every point.

Sensitivity Score	Variability Score	Worst-Case Gap
0.776	0.2541	0.8332

Figure 9: NN sensitivity and performance variance quantified

V.2.2 Bellman Held Karp Theoretical Runtime Analysis

Evident in the box plot, the empirical runtimes for BHK demonstrated minimal sensitivity to the varied input types. In contrast to heuristic algorithms like NN, BHK explored the entire problem space regardless of the worst-case design or perturbations. Consequently, comparing across input types using linear regression would prove uninformative. To address this, the empirical runtime was compared to a theoretical benchmark instead. This alternative analysis still provides insight towards the usefulness of worst-case performance in practice.

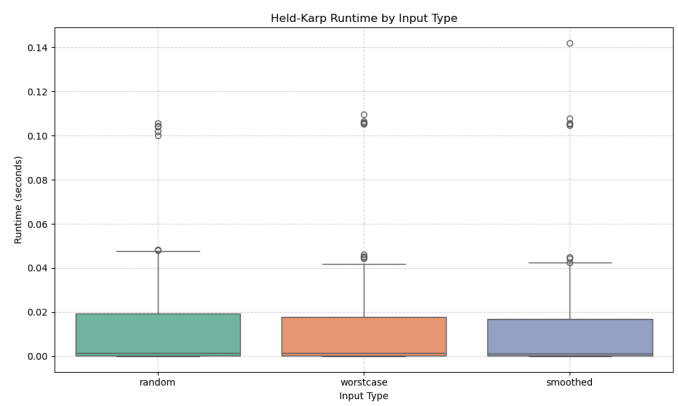

Figure 10: BHK runtime sensitivity to input type

Log Theoretical Runtime Regression

An expression for log theoretical runtime was defined in a function and applied on each input size. The theoretical runtime complexity for Held-Karp is known as O($n^2 2^n$). Thus, the runtime for a given input can be expressed as $c + n^2 2^n$.

By taking the log of this for a given input using $log(n^2 2^n) = 2log(n) + nlog(2)$ and applying it to each row in the dataframe, a log theoretical column was appended. The constant factor c was left out, as log-linear regression would be used to fit the trend and show relative growth behaviour instead. This involved performing a linear regression with log runtime as the dependent and log theoretical runtime as the independent variables.

Viewing the regression results, the slope of 0.169 is far less than 1, implying the experimental runtime increased slower than theoretical worst-case growth suggests. However, the R squared value of 0.585 means the theoretical complexity only explained around 59% of the variation in runtime. The remaining 41% due to possible implementation details, CPU, input structure, or the overhead not being captured by asymptotics.

Regression Statistics			
Multiple R	R Square	Adjusted R Square	Standard Error
0.764827	0.58496	0.58203732	0.576269299

	Coefficients	*Standard Error*	*t Stat*	*P-value*
Intercept	-1.7677578	0.169366865	-10.4374	2.8E-19
log_theoretical	0.16945821	0.011978439	14.1469	6.7E-29

Figure 11: BHK log theoretical runtime regression statistics

Plotting Log Theoretical Runtime Regression

Upon plotting the results, it is observed that for small log theoretical runtimes the observed log runtime starts below the predictive line. Yet as theoretical runtimes increase it grows faster and eventually exceeds the predicted values. The linear regression is underestimating the steepness of the curve, suggesting a non-linear fit may better capture behaviour.

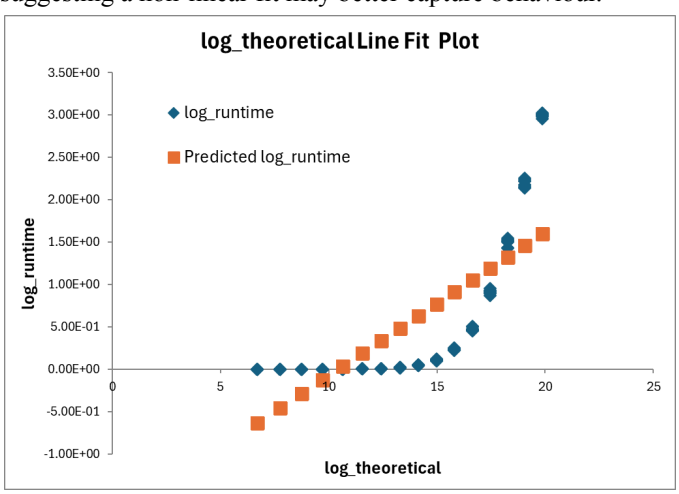

Figure 12: BHK log theoretical runtime regression scatter plot

Quantifying Results

These results indicate that for BHK, worst-case analysis may prove useful in practice for long-term growth, but predicts poorly for small instances. The runtime was consistently lower than predicted, and input type had negligible effect. This contrasts with NN, which was highly sensitive to input structure and initial state.

V.2.3 Christofides Approximation Analysis

Sensitivity to Input Structure

The attempts to construct worst-case inputs for Christofides surprisingly yielded better results than random input, with faster runtimes and relative error around 0.

input_type	mean_rel_error	std_rel_error	median_rel_error	max_rel_error
expected	0.018385	0.024465	0.009979	0.078978
worst_case_1	0.000000	0.000000	0.000000	0.000000
worst_case_2	0.000016	0.000035	0.000000	0.000079
worst_case_3	0.000000	0.000000	0.000000	0.000000

Figure 13: Christofides expected case against worst case statistics

When smoothing the worst-case instances, the error increased but remained beneath the expected case, with a difference of approximately 0.007 between it and the next highest smoothed case. Runtime proved similar, with expected-case edging slightly higher.

input_type	mean_rel_error	std_rel_error	median_rel_error	max_rel_error
expected	0.018385	0.024465	0.009979	0.078978
smoothed_case_1	0.000841	0.002658	0.000000	0.008405
smoothed_case_2	0.000019	0.000059	0.000000	0.000187
smoothed_case_3	0.011139	0.016716	0.000000	0.047045

Figure 14: Christofide expected case against smoothed case statistics

The results suggest Christofides performs worse on random input than the theoretical worst-case structures. But given the low input size and possible implementation issues regarding the worst-case input, no definitive conclusions can be made. However, the findings do support that Christofides is susceptible to adversarial inputs, although less compared to heuristics like NN. Given its low sensitivity, theoretical worst-case performance may prove more reflective of practical behaviour.

To investigate this, and maintain alignment with the project goal. The empirical performance of Christofides expected-case was compared to its theoretical guarantees, following a similar methodology to Bellman-Held-Karp.

One-Sample T-Test

A one-sample t-test was performed to compare the average relative error to Christofides 1.5 approximation ratio, or 0.5 error. The resulting p-value was $7.89 * 10^{57}$ which is effectively zero. As $p < 0.05$, The test shows the average relative error was significantly lower than the theoretical worst-case bound. Indicating that in practice it performs far better.

Input Size and Relative Error Regression

To investigate whether the low error was due to the low input size, a linear regression was performed. The 0.07 R squared value indicates only 7% of the error variation was explained by city size, the rest likely due to instance randomness. All p-values were insignificant, showing there is no strong

evidence that error increases linearly or quadratically with input size based on this instance.

Regression Statistics			
Multiple R	R Square	Adjusted R Square	Standard Error
0.27195687	0.07396	0.060441715	0.017555147

	Coefficients	Standard Error	t Stat	P-value
Intercept	-0.00894	0.012748502	-0.700961571	0.48452
cities	0.00151	0.002394574	0.632625419	0.52803
cities_squared	-1.3E-05	0.000102875	-0.126062922	0.89987

Figure 15: Christofides input size and relative error regression stats

Plotting Input Size and Relative Error Regression

The linear regression is visualized below, where the slope of the predicted relative error increases approximately 0.00151 per city. These results mean Christofides worst-case upper bound largely overestimates the error in solution quality compared to practice, particularly for small input sizes.

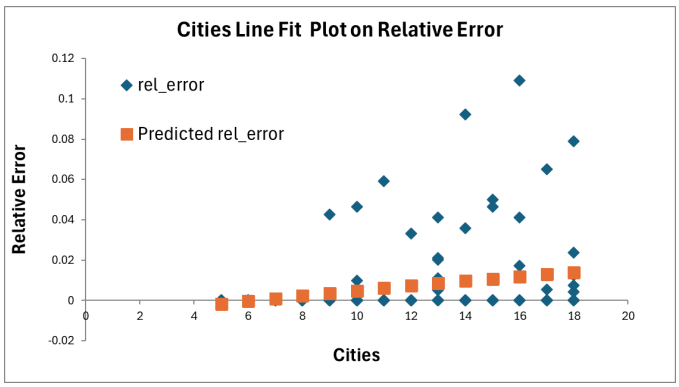

Figure 16: Christofides input size and relative error regression plot

V.3 Cross Comparative Analysis

As each algorithm was evaluated under different benchmarks and underwent analysis changes, directly comparing numerical results as planned were no longer suitable. The table below summarises the findings from each individual analysis, capturing the behaviour of the algorithm under each metric qualitatively instead.

Algorithm	Input Sensitivity	Error Variance	Worst-case Gap
NN	High	High	High
BHK	Low	None	Low
Christofides	Low-Moderate	Low	Moderate

Figure 17: Summary table of qualitative findings for input sensitivity and performance variance

Interpretation of Results

The results suggest a trend, where the algorithms that were more input sensitive with higher error variance had a greater worst-case gap. Indicating they had practical performance less reflected by worst-case analysis. Thus, it could be deduced that worst-case usefulness is a function of input sensitivity and performance variance. Where small changes in input lead to large variations in performance, causing loose worst-case upper bounds.

Quantifying Worst-Case Usefulness

A model can be proposed to quantify the usefulness of worst-case analysis according to this behaviour, where the input sensitivity and performance variation is represented by the metrics defined in the methodology.

$$Usefulness\ of\ WorstCase = 1 + WCG + VS + SS$$

Plugging in the values calculated for each variable in the Quantifying Results section for the NN Analysis, produces a worst-case usefulness score of 0.349 as shown.

$$Usefulness\ of\ WorstCase = 1 + 0.8332 + 0.2541 + 0.7761$$
$$= \frac{1}{2.86331} = 0.349$$

A score of 1 indicates a perfect alignment with worst-case performance. Thus, a score of 0.349 appropriately reflects the extent to which worst-case complexity theory misrepresented the expected performance of NN. The simple model could prove informative for comparing worst-case usefulness across different algorithms. Where lower worst-case gap, input sensitivity, and variability leads to higher worst-case accuracy.

VI. Conclusions and Future Work

This project explored how NP-hard problems are solved efficiently in practice despite its worst-case performance, using the Travelling Salesman Problem as a case study. Based on the results, worst-case analysis appears most useful for algorithms with stable performance and low sensitivity to small input changes, particularly at larger input sizes. This was most evident in the Bellman-Held-Karp (BHK) algorithm, which aligned the best with its theoretical runtime and remained unchanged under varying input types. In contrast, worst-case analysis proved heavily misleading for the Nearest Neighbor (NN) heuristic, whose performance varied greatly with input structure, demonstrated through both constructed adversarial inputs and smoothed analysis.

A key limitation of this study was the difficulty quantifying the usefulness of worst-case analysis numerically. The varied nature of each algorithm made a universal metric hard to define. Especially since without perfectly constructed worst-case instances obtaining conclusive empirical results proved challenging. Which led to the alternative analysis for BHK and Christofides.

Future work could build on these findings to produce a formal model that can quantify the usefulness of worst-case performance for a given algorithm and NP-hard problem. It would require improving the metrics used to represent input sensitivity and performance variation, ensuring they cover the varied behaviour of each algorithm type. Applying this to the model proposed in this paper, could lead to an effective quantification of when worst-case analysis is representative, and when it fails to capture real world-behavior.

Reference List

1. Arora, S. (1996, October). Polynomial time approximation schemes for Euclidean TSP and other geometric problems. In Proceedings of 37th Conference on Foundations of Computer Science (pp. 2-11). IEEE.

2. Chekuri, C. (2018). Approximation Algorithms: Introduction. Illinois.

3. Christofides, N. (2022, March). Worst-case analysis of a new heuristic for the travelling salesman problem. In Operations Research Forum (Vol. 3, No. 1, p. 20). Cham: Springer International Publishing

4. Cornell University. (2012). Lecture Notes on Approximation Algorithms. New York.

5. Genova, K., & Williamson, D. P. (2017). An experimental evaluation of the best-of-many Christofides' algorithm for the traveling salesman problem. Algorithmica, 78(4), 1109-1130.

6. Hougardy, S., & Wilde, M. (2015). On the nearest neighbor rule for the metric traveling salesman problem. Discrete Applied Mathematics, 195, 101-103.

7. Johnson, D. S. (1973, April). Approximation algorithms for combinatorial problems. In Proceedings of the fifth annual ACM symposium on Theory of computing (pp. 38-49).

8. Karlin, A. R., Klein, N., & Gharan, S. O. (2021, June). A (slightly) improved approximation algorithm for metric TSP. In Proceedings of the 53rd Annual ACM SIGACT Symposium on Theory of Computing (pp. 32-45).

9. Kundakcioglu, O. E., & Yurtkuran, A. (2023). A comprehensive survey on the generalized traveling salesman problem: Formulations, approaches, and future directions. European Journal of Operational Research, 312(1), 1–32. https://doi.org/10.1016/j.ejor.2023.06.022

10. Mouatadid, L. (2016). Introduction to Approximation Algorithms. Toronto.

11. Spielman, D. A., & Teng, S. H. (2004). Smoothed analysis of algorithms: Why the simplex algorithm usually takes polynomial time. Journal of the ACM (JACM), 51(3), 385-463.

Vazirani, V. V. (2001). Approximation algorithms (Vol. 1). Berlin: springer.

Github Repository

https://github.cs.adelaide.edu.au/a1926920/topics